

RMSProp: AdaGrad That Forgets

Contents

1. Overview	2
2. Prerequisites	2
3. AdaGrad's problem, in one formula	3
4. The fix: an average that forgets	3
5. Convex and non-convex losses	5
5.1 AdaGrad against RMSProp on MNIST	5
6. Strengths and limits	6
7. RMSProp in Keras	7
8. Summary	8
9. Sources	8
10. Key terms	8

1. Overview

Key point: RMSProp replaces AdaGrad's sum of squared gradients with an exponentially weighted moving average of them: $v_t = \beta v_{t-1} + (1 - \beta)(\nabla L)^2$. Old gradients fade away, so v_t cannot grow without limit and the learning rate does not shrink to nothing.

RMSProp, short for *root mean square propagation* (Hinton 2012, lecture 6e), is an improvement of [AdaGrad](#). It keeps AdaGrad's good idea, a separate learning rate for every parameter, and removes its weakness: learning rates that only ever fall.

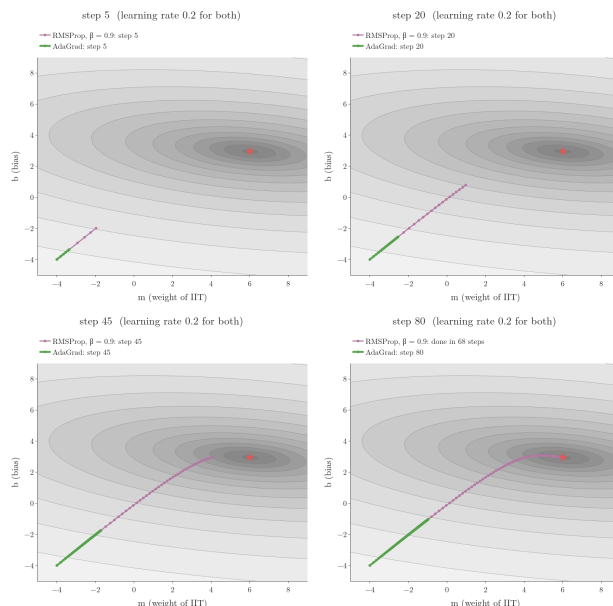


Figure 1: AdaGrad and RMSProp with the same learning rate 0.2 on the elongated bowl of a sparse feature. AdaGrad's steps keep shrinking and it crawls; RMSProp keeps moving and reaches the minimum

Figure 1 shows the difference on the students data of the AdaGrad Note. With the same learning rate, RMSProp brings the loss within 0.01 of its minimum in 68 steps; AdaGrad has not got there after 300.

2. Prerequisites

- The [AdaGrad Note](#): per-parameter learning rates $\eta/(\sqrt{v_t} + \epsilon)$, the elongated bowl, and the shrinking learning rate.
- The [EWMA Note](#): $V_t = \beta V_{t-1} + (1 - \beta)\theta_t$ and why old values fade.

3. AdaGrad's problem, in one formula

Key point: AdaGrad's v_t adds up every squared gradient since the first step. After many steps v_t is huge, $\eta/\sqrt{v_t}$ is tiny, and the updates almost stop.

Recall the data: 100 students, the sparse IIT feature (mostly 0) and the package as target. Its loss is an elongated bowl over the weight m and the bias b , and AdaGrad heads for the minimum by giving each parameter its own learning rate. But AdaGrad divides η by $\sqrt{v_t}$, where

$$v_t = v_{t-1} + (\nabla L)^2$$

is the sum of all past squared gradients. Every step adds a positive amount, so v_t only grows. After many steps the learning rate of a parameter such as b becomes so small that its updates are almost zero, and AdaGrad stops short of the minimum.

With $\eta = 0.2$, AdaGrad's v for b grows from 253 after the first step to 2,171 after 10 steps and 20,654 after 300. Its learning rate $0.2/\sqrt{v}$ falls from 0.0126 to 0.0014, and after 300 steps (m, b) is only at (1.50, 1.22), far from the best (6.01, 2.96) (Notebook).

The whole problem comes from one fact: v_t uses every gradient of the past, so it can only grow. We need a v_t that does not grow without limit.

4. The fix: an average that forgets

Key point: RMSProp makes v_t an EWMA of the squared gradients. Recent gradients count most and old ones fade, so v_t follows the current size of the gradients instead of piling up history.

RMSProp changes only the line that computes v_t :

1. **In words:** keep an exponentially weighted moving average of each parameter's squared gradient, then divide the learning rate by its square root, as AdaGrad does.
2. **Formula:**

$$v_t = \beta v_{t-1} + (1 - \beta) (\nabla L(w_t))^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \nabla L(w_t)$$

with $v_0 = 0$ and β usually 0.9 (Hinton 2012).

3. **Example:** three squared gradients $g_1^2 = 9$, $g_2^2 = 4$, $g_3^2 = 1$ with $\beta = 0.9$:

$$v_1 = 0.1 \times 9 = 0.9, \quad v_2 = 0.9 \times 0.9 + 0.1 \times 4 = 1.21, \quad v_3 = 0.9 \times 1.21 + 0.1 \times 1 = 1.189$$

AdaGrad's sum would be $9 + 4 + 1 = 14$ (Notebook).

Unrolling the average, as in the [EWMA Note](#), shows the weights:

$$v_3 = (1 - \beta) (\beta^2 g_1^2 + \beta g_2^2 + g_3^2)$$

The oldest squared gradient is multiplied by β^2 , the newest by 1. Since $\beta < 1$, old gradients are forgotten little by little and recent ones count most. So v_t never shoots up: it stays about the size of the recent squared gradients, the learning rate does not become tiny, and the updates continue.

RMSProp modifies AdaGrad by changing the gradient accumulation into an exponentially weighted moving average; it discards history from the extreme past so that it can converge rapidly after finding a convex bowl, as if it were an instance of AdaGrad started inside that bowl (Goodfellow et al. 2016, §8.5.2).

The name says what the method does: the denominator $\sqrt{v_t}$ is the **root** of a (weighted) **mean** of the **squared** gradients.

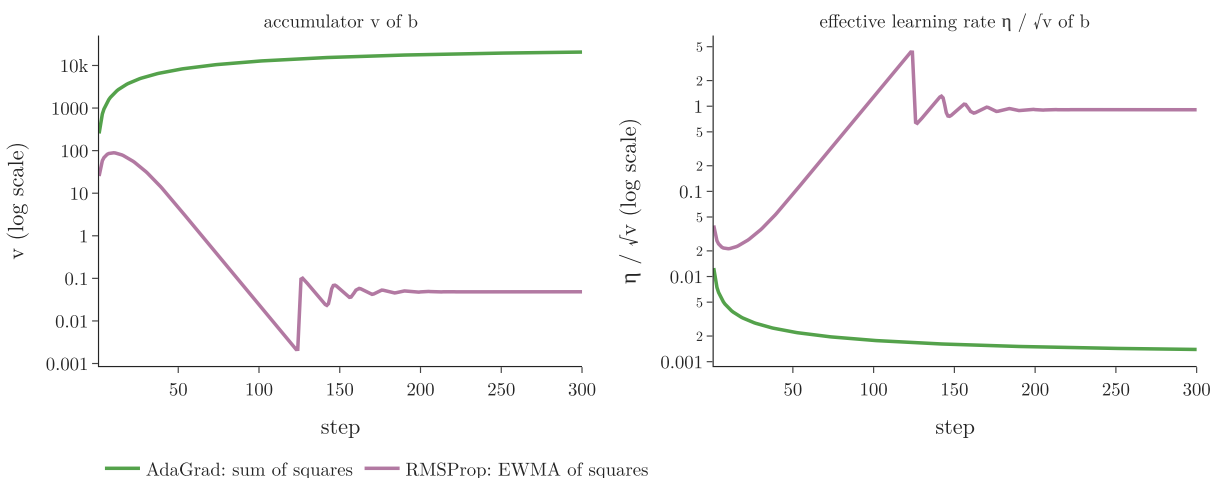


Figure 2: The accumulator v of the bias b and its effective learning rate $\eta/\sqrt{v}$, learning rate 0.2. AdaGrad’s sum (green) only grows, so its learning rate only falls. RMSProp’s average (purple) falls again as the gradients shrink near the minimum

Figure 2 shows both accumulators for b . AdaGrad’s v keeps growing; RMSProp’s rises at the start, when the gradients are large, then falls as they shrink. RMSProp’s learning rate for b stays useful, and the path of Figure 1 reaches the minimum.

Extra: RMSProp’s step is $\eta g / \sqrt{v}$, and $\sqrt{v}$ is about the size of the recent gradients, so near the minimum each step stays roughly of size η even when the gradients are tiny. With a constant learning rate the weights then keep jittering around the minimum: in the last 100 of 300 steps RMSProp stays within 0.11 of the best (m, b) but does not settle exactly (Notebook). A small η , such as Keras’ default 0.001, or a learning-rate schedule (see the [stochastic gradient descent Note](#)) keeps the jitter small.

5. Convex and non-convex losses

Key point: On a simple convex bowl, AdaGrad can do fine with a large enough learning rate. Its shrinking learning rate hurts most in deep networks, whose losses are non-convex. RMSProp works in both.

On a convex problem such as linear regression, AdaGrad and RMSProp can follow very similar paths: with a large enough learning rate AdaGrad converges, as in the [AdaGrad Note](#). The difference shows in neural networks. There the path crosses many different regions before reaching a bowl, and AdaGrad may have shrunk its learning rate too much before it gets there (Goodfellow et al. 2016, §8.5.2).

5.1 AdaGrad against RMSProp on MNIST

Key point: Same network, same learning rate 0.001, only the accumulator differs. After 30 epochs AdaGrad's training loss is still 0.24; RMSProp's is 0.0001.

The data is the MNIST handwritten digits (see the [MNIST Note](#)): 10,000 training images, each with 784 pixel **features** (input variables) and the digit as **target** (the output we predict), and the 10,000 test images for validation. The network has hidden layers of 128 and 64 ReLU nodes. Both optimizers use learning rate 0.001 (Keras' default for both), batch size 64, accumulators starting at 0, and 30 epochs, with 3 seeds each. After every epoch we record the median effective learning rate $\eta/\sqrt{v}$ over all weights.

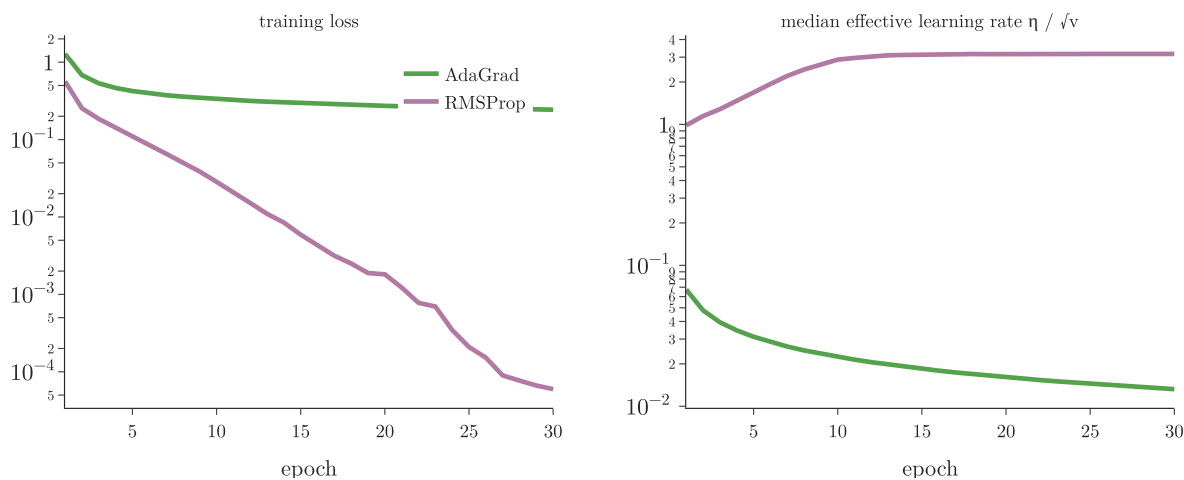


Figure 3: MNIST, mean of 3 seeds. Left: training loss per epoch. Right: the median effective learning rate. AdaGrad's (green) falls epoch after epoch and its loss levels off; RMSProp's (purple) does not fall, and its loss keeps dropping

Figure 3 and the Notebook give:

	AdaGrad	RMSProp
Training loss, epoch 1	1.28	0.56
Training loss, epoch 10	0.33	0.029
Training loss, epoch 30	0.24	0.0001
Validation accuracy, epoch 30	0.920	0.956
Median $\eta/\sqrt{v}$, epoch 1 $\rightarrow$ 30	0.067 $\rightarrow$ 0.013	0.99 $\rightarrow$ 3.2

AdaGrad’s median learning rate falls by a factor of 5 over the 30 epochs, and its loss levels off: it drops from 0.33 to only 0.24 between epochs 10 and 30. RMSProp’s median learning rate does not fall at all; it rises as the gradients become small near the minimum, and its training loss keeps dropping, to 0.0001. Validation accuracy follows: 0.956 against 0.920 (Notebook).

Extra: A larger learning rate helps AdaGrad but does not remove the gap. With $\eta = 0.01$, 0.03 and 0.1 (seed 0), its training loss after 30 epochs is 0.016, 0.0014 and 0.0013, still above RMSProp’s 0.0001 at $\eta = 0.001$. Its validation accuracy at $\eta = 0.03$, 0.961, is even a little above RMSProp’s 0.956: with a well-chosen learning rate AdaGrad does well on this small network, in line with AdaGrad performing well for some deep learning models but not all (Goodfellow et al. 2016, §8.5.1) (Notebook).

6. Strengths and limits

Key point: RMSProp is an effective, widely used optimizer for deep networks. RMSProp was the usual choice before Adam, and it is a natural next try when Adam does not give good results.

RMSProp has been shown empirically to be an effective and practical optimization algorithm for deep neural networks, and it is one of the go-to methods of deep learning practitioners (Goodfellow et al. 2016, §8.5.2). Before Adam appeared, it was the optimizer most networks were trained with. It still competes with Adam: if Adam does not give good results on a problem, RMSProp is a natural next try.

Two limits are worth knowing:

- Compared with AdaGrad, it adds a hyperparameter, β , which sets how long the average remembers (Goodfellow et al. 2016, §8.5.2).
- Its average starts at $v_0 = 0$ with no correction, so early in training v_t is pulled towards 0 (Goodfellow et al. 2016, §8.5.3), the start-up effect seen in the [EWMA Note](#).

Adam fixes the second and adds momentum, and usually performs a little better, which is why RMSProp is used less today (see the [Adam Note](#)).

Extra: RMSProp was never published as a paper; it comes from Hinton’s 2012 Coursera lecture slides, which present it as a mini-batch version of an older method, rprop, and suggest $\beta = 0.9$ (Hinton 2012, lecture 6e; Ruder 2016, §4.5). A paper-free origin is why

it is usually cited as “Hinton 2012”.

7. RMSProp in Keras

Key point: `keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)`. Keras calls β rho.

Python: RMSProp in Keras.

```
import keras
opt = keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)
model.compile(optimizer=opt, loss="sparse_categorical_crossentropy",
               metrics=["accuracy"])
```

The defaults are `learning_rate=0.001`, `rho=0.9` and `epsilon=1e-7`, added inside the square root (Keras RMSprop documentation).

8. Summary

	AdaGrad	RMSProp
v_t	$v_{t-1} + g_t^2$ (sum of all)	$\beta v_{t-1} + (1 - \beta)g_t^2$ (EWMA)
Old gradients	count forever	fade by β per step
v_t over time	only grows	follows the recent gradient size
Learning rate $\eta/\sqrt{v_t}$	only falls	can recover
Students data, $\eta = 0.2$	not there after 300 steps	68 steps
MNIST, 30 epochs, $\eta = 0.001$	loss 0.24	loss 0.0001

- RMSProp is AdaGrad with the sum of squared gradients replaced by their EWMA.
- Old gradients fade, so the accumulator stays the size of the recent gradients and the learning rate does not collapse.
- It keeps AdaGrad's per-parameter learning rates, so it still handles elongated bowls and sparse features.
- It works well in deep networks and was the standard choice before Adam; usual values $\beta = 0.9$, $\eta = 0.001$.

9. Sources

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. §8.5.2 RMSProp (algorithm 8.5).
- Hinton, G. (2012). *Neural Networks for Machine Learning*, Coursera, lecture 6e: rmsprop: divide the gradient by a running average of its recent magnitude (slides).
- Ruder, S. (2016). An overview of gradient descent optimization algorithms. arXiv:1609.04747. §4.5 RMSprop.
- Keras API documentation: **RMSprop** optimizer, keras.io/api/optimizers/rmsprop.

10. Key terms

Term	Meaning
RMSProp	AdaGrad with an EWMA of squared gradients in place of their sum: $v_t = \beta v_{t-1} + (1 - \beta)g_t^2$
Root mean square	The square root of the average of squared values: the typical size of the gradients
Accumulator v_t	The running record of squared gradients that divides the learning rate
rho	Keras' name for RMSProp's decay factor β ; default 0.9
Feature	An input variable, such as one pixel of an image

Term	Meaning
Target	The output we predict, such as the digit